

Teach Module 35 README

Module 35: Frontend to API Integration

This module focuses on connecting a React frontend to a Java Spring Boot SOAP web service.

Topics covered:

- SOAP consumption with `fetch` or `axios`
- XML request and response handling
- Async handling and promises
- Loading, error, empty, and success states
- API bindings and frontend service layers
- Mapping backend XML into frontend-friendly JavaScript objects

Core idea:

Keep React components focused on UI, keep API logic in a service layer, and convert backend XML into clean frontend data before rendering.

1. What Frontend-to-API Integration Means

A React frontend often needs to communicate with a backend service. In a typical flow, the user performs an action, the frontend sends an HTTP request, the backend processes it, and the frontend displays the result.

For this module, the backend is a SOAP-style Java Spring Boot service. That means the frontend may send and receive XML instead of JSON.

Typical flow:

```
React component
-> frontend service function
-> HTTP request using fetch or axios
-> Spring Boot SOAP endpoint
-> XML SOAP response
-> parse XML
-> map to JavaScript object
-> render UI
```

The important skill is not just making the HTTP call. A production-quality frontend also needs to handle waiting, errors, missing data, and response mapping.

2. SOAP vs REST

REST APIs commonly exchange JSON.

Example JSON request:

```
{
  "customerId": 101
}
```

SOAP APIs exchange XML, usually wrapped in a SOAP envelope.

Example SOAP request:

```
<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Body>
    <getCustomerRequest>
      <customerId>101</customerId>
    </getCustomerRequest>
  </soapenv:Body>
</soapenv:Envelope>
```

When consuming SOAP from React, you still use normal browser HTTP tools such as `fetch` or `axios`. The difference is that the request body and response format are XML.

3. Calling a SOAP API with `fetch`

Example service function:

```
export async function getCustomerById(customerId) {
  const soapBody = `
    <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
      <soapenv:Body>
        <getCustomerRequest>
          <customerId>${customerId}</customerId>
        </getCustomerRequest>
      </soapenv:Body>
    </soapenv:Envelope>
  `;

  const response = await fetch("http://localhost:8080/ws", {
    method: "POST",
    headers: {
      "Content-Type": "text/xml"
    },
    body: soapBody
  });

  if (!response.ok) {
    throw new Error(`API request failed: ${response.status}`);
  }

  const xmlText = await response.text();
  return parseCustomerResponse(xmlText);
}
```

For XML responses, use:

```
await response.text();
```

Do not use:

```
await response.json();
```

unless the backend truly returns JSON.

4. Parsing XML in JavaScript

The browser includes `DOMParser`, which can convert XML text into a traversable XML document.

```
function parseCustomerResponse(xmlText) {
  const parser = new DOMParser();
  const xmlDoc = parser.parseFromString(xmlText, "text/xml");

  const id = xmlDoc.getElementsByTagName("id")[0]?.textContent;
  const firstName = xmlDoc.getElementsByTagName("firstName")[0]?.textContent;
  const lastName = xmlDoc.getElementsByTagName("lastName")[0]?.textContent;
  const email = xmlDoc.getElementsByTagName("email")[0]?.textContent;
  const status = xmlDoc.getElementsByTagName("status")[0]?.textContent;

  if (!id) {
    return null;
  }

  return {
    id: Number(id),
    fullName: `${firstName} ${lastName}`,
    email,
    isActive: status === "ACTIVE"
  };
}
```

The goal is to turn backend XML into frontend-friendly data.

Backend-style XML:

```
<customer>
  <id>101</id>
  <firstName>Ava</firstName>
  <lastName>Patel</lastName>
  <email>ava.patel@example.com</email>
  <status>ACTIVE</status>
</customer>
```

Frontend object:

```
{
  id: 101,
  fullName: "Ava Patel",
  email: "ava.patel@example.com",
```

```
    isActive: true
  }
```

5. Async Handling and Promises

API requests take time. React must handle the waiting period and possible failure.

A good component usually tracks:

- loading
- error
- returned data
- whether a search has been attempted

Example:

```
import { useEffect, useState } from "react";
import { getCustomerById } from "../services/customerService";

export default function CustomerProfile({ customerId }) {
  const [customer, setCustomer] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");

  useEffect(() => {
    async function loadCustomer() {
      try {
        setLoading(true);
        setError("");

        const data = await getCustomerById(customerId);
        setCustomer(data);
      } catch (err) {
        setCustomer(null);
        setError("Could not load customer details.");
      } finally {
        setLoading(false);
      }
    }

    if (customerId) {
      loadCustomer();
    }
  }, [customerId]);

  if (loading) return <p>Loading customer...</p>;
  if (error) return <p role="alert">{error}</p>;
  if (!customer) return <p>No customer selected.</p>;

  return (
    <section>
      <h2>{customer.fullName}</h2>
    </section>
  );
}
```

```
    <p>{customer.email}</p>
  </section>
);
}
```

6. Frontend Service Layer

Avoid placing SOAP XML construction, HTTP calls, and XML parsing directly inside React components.

Better structure:

```
src/
  components/
    CustomerLookup.jsx
  services/
    customerService.js
```

The component should handle:

- form input
- button clicks
- loading states
- error messages
- rendering results

The service should handle:

- building SOAP XML
- sending HTTP requests
- checking HTTP errors
- parsing XML
- mapping data

This keeps the code easier to test, reuse, and maintain.

7. Error States and User Feedback

APIs can fail for many reasons:

- backend server is down
- endpoint URL is wrong
- network request fails
- CORS is not configured
- server returns 500
- SOAP fault is returned
- XML response is missing expected fields

Good UI should not crash or remain blank. It should show clear user-facing feedback.

Examples:

```
Loading customer...
No customer found.
```

```
Could not connect to the server.  
Unable to load customer details.  
Please enter a customer ID.
```

Detailed technical errors can be logged for developers, but the UI should use plain, helpful messages.

8. Practice Exercises

Exercise 1: Basic SOAP Request

Create a React button that calls a SOAP endpoint using `fetch`.

Practice:

- `POST` request
- `Content-Type: text/xml`
- sending a SOAP envelope
- reading the response with `response.text()`

Goal: display the raw XML response on the page.

Exercise 2: XML Parsing

Take a SOAP XML response and extract fields using `DOMParser`.

Example fields:

- `customerId`
- `firstName`
- `lastName`
- `email`
- `status`

Goal: convert XML into a clean JavaScript object.

Exercise 3: Customer Search UI

Build a small `CustomerLookup` component.

User flow:

1. User enters a customer ID.
2. User clicks Search.
3. React calls the SOAP API.
4. UI shows customer details.

Include:

- loading message
- error message
- no-result state
- successful result view

Exercise 4: Frontend Service Layer

Move all API logic out of the React component.

Create:

```
src/
  components/
    CustomerLookup.jsx
  services/
    customerService.js
```

Goal: the component should not build SOAP XML or parse XML directly.

Exercise 5: Error Handling

Simulate different failures.

Handle:

- server returns `500`
- network request fails
- customer ID is empty
- XML response is missing fields
- SOAP fault response is returned

Goal: show user-friendly messages instead of crashing the UI.

Exercise 6: Axios Version

Repeat the same API call using `axios` instead of `fetch`.

Practice:

- `axios.post`
- custom headers
- reading XML response
- handling errors with `try/catch`

Exercise 7: JSON Mapping

Convert backend-style XML into frontend-friendly data.

Example XML:

```
<firstName>Ava</firstName>
<lastName>Patel</lastName>
<accountStatus>ACTIVE</accountStatus>
```

Mapped object:

```
{
  fullName: "Ava Patel",
  isActive: true
}
```

Goal: keep the UI independent from backend field names.

Exercise 8: Reusable SOAP Helper

Create a generic helper function:

```
sendSoapRequest(url, soapBody)
```

Then reuse it for multiple service calls:

```
getCustomerById(id);  
getOrdersByCustomerId(customerId);  
getAccountStatus(customerId);
```

Goal: avoid repeating fetch or axios setup everywhere.

Exercise 9: Mini Dashboard

Build a page that loads multiple API sections:

- customer profile
- recent orders
- account status

Each section should have its own:

- loading state
- error state
- success state

Goal: practice managing multiple async API calls in one UI.

Exercise 10: Integration Lab

Build a complete frontend integration flow:

```
Search customer  
-> call SOAP API  
-> parse XML  
-> map data  
-> render customer details  
-> fetch customer orders  
-> show loading/error/success states
```

Best starter combination:

- Exercise 3
- Exercise 4
- Exercise 5

Together, these cover React UI, service-layer separation, async behavior, and error handling.

9. Lab: Customer Lookup with SOAP API

Goal

Build a React screen that searches for a customer by ID, calls a SOAP-style API, parses XML, maps it into JavaScript data, and displays loading, error, empty, and success states.

Scenario

You are building a customer support dashboard. A support user enters a customer ID and retrieves customer information from a Spring Boot SOAP service.

Files to Create

```
src/
  components/
    CustomerLookup.jsx
  services/
    customerService.js
```

Step 1: Create the Service Layer

Create `src/services/customerService.js`.

```
export async function getCustomerById(customerId) {
  const soapBody = `
    <soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
      <soapenv:Body>
        <getCustomerRequest>
          <customerId>${customerId}</customerId>
        </getCustomerRequest>
      </soapenv:Body>
    </soapenv:Envelope>
  `;

  const response = await fetch("http://localhost:8080/ws", {
    method: "POST",
    headers: {
      "Content-Type": "text/xml"
    },
    body: soapBody
  });

  if (!response.ok) {
    throw new Error("Customer service request failed");
  }

  const xmlText = await response.text();
  return parseCustomerResponse(xmlText);
}

function parseCustomerResponse(xmlText) {
  const xmlDoc = new DOMParser().parseFromString(xmlText, "text/xml");

  const fault = xmlDoc.getElementsByTagName("faultstring")[0]?.textContent;
```

```

if (fault) {
  throw new Error(fault);
}

const id = xmlDoc.getElementsByTagName("id")[0]?.textContent;
const firstName = xmlDoc.getElementsByTagName("firstName")[0]?.textContent;
const lastName = xmlDoc.getElementsByTagName("lastName")[0]?.textContent;
const email = xmlDoc.getElementsByTagName("email")[0]?.textContent;
const status = xmlDoc.getElementsByTagName("status")[0]?.textContent;

if (!id) {
  return null;
}

return {
  id: Number(id),
  fullName: `${firstName} ${lastName}`,
  email,
  isActive: status === "ACTIVE"
};
}

```

Step 2: Create the React Component

Create `src/components/CustomerLookup.jsx` .

```

import { useState } from "react";
import { getCustomerById } from "../services/customerService";

export default function CustomerLookup() {
  const [customerId, setCustomerId] = useState("");
  const [customer, setCustomer] = useState(null);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState("");
  const [searched, setSearched] = useState(false);

  async function handleSearch(event) {
    event.preventDefault();

    if (!customerId.trim()) {
      setError("Please enter a customer ID.");
      setCustomer(null);
      return;
    }

    try {
      setLoading(true);
      setError("");
      setSearched(true);

      const result = await getCustomerById(customerId.trim());

```

```

        setCustomer(result);
    } catch (err) {
        setCustomer(null);
        setError("Unable to load customer details.");
    } finally {
        setLoading(false);
    }
}

return (
  <main>
    <h1>Customer Lookup</h1>

    <form onSubmit={handleSearch}>
      <label htmlFor="customerId">Customer ID</label>
      <input
        id="customerId"
        value={customerId}
        onChange={(event) => setCustomerId(event.target.value)}
        placeholder="Enter customer ID"
      />
      <button type="submit" disabled={loading}>
        Search
      </button>
    </form>

    {loading && <p>Loading customer...</p>}

    {error && <p role="alert">{error}</p>}

    {!loading && !error && searched && !customer && (
      <p>No customer found.</p>
    )}

    {!loading && !error && customer && (
      <section>
        <h2>{customer.fullName}</h2>
        <p>Email: {customer.email}</p>
        <p>Status: {customer.isActive ? "Active" : "Inactive"}</p>
      </section>
    )}
  </main>
);
}

```

Step 3: Render the Component

In `App.jsx` :

```
import CustomerLookup from "../components/CustomerLookup";
```

```
export default function App() {  
  return <CustomerLookup />;  
}
```

Step 4: Test with Sample XML

If the backend is not running yet, temporarily replace the `fetch` call with this XML string:

```
const xmlText = `  
  <soapenv:Envelope>  
    <soapenv:Body>  
      <getCustomerResponse>  
        <customer>  
          <id>101</id>  
          <firstName>Ava</firstName>  
          <lastName>Patel</lastName>  
          <email>ava.patel@example.com</email>  
          <status>ACTIVE</status>  
        </customer>  
      </getCustomerResponse>  
    </soapenv:Body>  
  </soapenv:Envelope>  
`;  
  
return parseCustomerResponse(xmlText);
```

Step 5: Required Outcomes

The lab is complete when:

- the form prevents empty searches
- the service sends a SOAP XML request
- the response is parsed with `DOMParser`
- XML is mapped to a JavaScript object
- the UI handles loading, error, no-result, and success states
- API logic is separated from UI logic

10. Challenge Extension

Add a second button: `Load Orders`.

Create another service function:

```
getOrdersByCustomerId(customerId);
```

Display:

- order ID
- order date
- order total
- order status

This gives a more realistic frontend integration pattern: one screen calling multiple backend operations.

11. Common Mistakes to Avoid

- Calling `response.json()` on an XML response
- Putting SOAP request construction directly inside React components
- Forgetting loading and error states
- Assuming every XML tag always exists
- Ignoring CORS configuration during local development
- Exposing raw backend errors directly to users
- Letting backend XML field names control the entire frontend UI model

12. Quick Review

You should now be able to explain:

- why SOAP APIs use XML envelopes
- how to send a SOAP request from React
- how to parse XML with `DOMParser`
- why frontend service layers are useful
- how to handle async loading and error states
- how to map backend responses into clean frontend objects